

# Kardal Payment Integration Guide

What a merchant must build to accept **KHQR** and **Card** payments via the Kardal gateway

Reference app: `kardal-merchant-demo` (Laravel + React) · Gateway: `{base}/api/mch/v2/gateway` · Environment: Kardal DEV

**Scope.** Integration only. Assumes the merchant is already onboarded & **activated**, and holds its credential package ( `credential.txt` + `*-public.key` ). This guide covers what *you*, the integrating merchant, build: auth, signing, checkout (KHQR + Card), status polling, and the notify webhook.

## 1. What the merchant must do — checklist

| # | Responsibility                                                                                                                                                                      | Where                    |
|---|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------|
| 1 | Store credentials server-side (never in browser): <code>client_id/secret</code> , <code>username/password</code> , <code>seller_code</code> , <code>api_key</code> , RSA public key | Backend config / secrets |
| 2 | Get + cache an OAuth access token; attach <code>Bearer</code> to every gateway call                                                                                                 | Backend                  |
| 3 | Build + attach the request <code>sign</code> (MD5 / HMAC-SHA256) to every gateway request                                                                                           | Backend                  |
| 4 | Create the order: KHQR ( <code>nativePay</code> ) or Card ( <code>directPay</code> ) with a unique <code>out_trade_no</code>                                                        | Backend                  |
| 5 | Render the result to the buyer: KHQR image, or the 3DS <code>html_confirm_payment</code> for cards                                                                                  | Frontend                 |
| 6 | Expose a public <code>notify_url</code> webhook; <b>verify its sign</b> ; mark the order paid                                                                                       | Backend                  |
| 7 | Reconcile via <code>queryOrder</code> (poll / fallback) — never trust the browser for final state                                                                                   | Backend                  |
| 8 | Be idempotent: dedupe by <code>out_trade_no</code> ; return HTTP 200 to the webhook                                                                                                 | Backend                  |

**Golden rule.** The buyer's browser is never the source of truth for payment success. Treat an order as paid only when the **notify webhook** (sign-verified) or **queryOrder** says `SUCCESS` .

## 2. Auth & signing (every request)

### 2.1 OAuth token

```
POST {base}/oauth/token
{ grant_type:"password", client_id, client_secret, username, password }
→ { access_token, expires_in:1800, refresh_token }
```

Cache the token (TTL < 1800s). Send `Authorization: Bearer {access_token}` on every `/api/mch/v2/gateway` call.

### 2.2 Signature

- Take the request params; drop `sign` , empty/null, and array/object values.
- Sort keys ascending; build `k=v&k2=v2...` .
- Append `&key={api_key}` .
- Hash: MD5 (default) or HMAC-SHA256 . Put result in `sign` ; set `sign_type` accordingly.

The same algorithm verifies the inbound notify (§5). Payload is sent **flat** (`seller_code`, `out_trade_no`, ... at top level) and signed flat.

## 3. KHQR checkout (nativePay)

### Request

```
POST {base}/api/mch/v2/gateway // Bearer + sign
{ service:"webpay.acquire.nativePay", service_code:"DYNAMICBANK",
  seller_code, out_trade_no, body, total_amount, currency:"USD",
  notify_url, redirect_url, sign_type:"MD5", sign }
```

### Response (data)

```
{ qrcode:"00020101...630448F7", expires_in:10799,
  order_info:{ token, out_trade_no, status:"WAITING" } }
```

## Flow

```
Buyer      Merchant FE      Merchant BE      Kardal
| checkout KHQR |      |      |      |
|----->| POST /payment/khqr ----->| nativePay -->|
|      |      |<-- qrcode -----|
|<-- render QR ---|      |      |
| scan w/ bank app ----- (pays at bank)
|      | poll /status -->| queryOrder ->|
|      |<-- SUCCESS -----|<-- SUCCESS ---|
|      | notify webhook <----- (server push)
```

- Render `qrcode` as a QR image (any KHQR-capable bank app scans it).
- While `WAITING` : poll `queryOrder` every ~3s (and/or wait for notify). Stop on terminal status or `expires_in` .
- **DYNAMICBANK** is this merchant's enabled KHQR BIC — send it as `service_code` (not `KHQR` ).

## 4. Card checkout (directPay + 3DS)

### Request

```
POST {base}/api/mch/v2/gateway // Bearer + sign
{ service:"webpay.acquire.directPay", service_code:"VISA_MASTER",
  seller_code, out_trade_no, body, total_amount, currency,
  card: hex(RSA({number, securityCode, expiry:{month,year}})),
  customer: hex(RSA({first_name,last_name,email,phone_number,phone_code})),
  holder_name, ip_address, notify_url, redirect_url, sign_type, sign }
```

### Response (data)

```
{ required_3ds:true, pre_card_input:true,
  html_confirm_payment:"<html>...auto-submitting 3DS form...</html>",
  order_info:{ out_trade_no, status:"WAITING" } }
```

### Rules

- **Encrypt card + customer server-side** with the merchant **public key**, hex-encoded. Raw PAN never touches your DB or the browser payload.
- If `html_confirm_payment` is non-empty: render that HTML in the browser — it auto-posts to the bank ACS for 3DS. The bank returns via the embedded `TermUrl` .
- Confirm final state via `notify` / `queryOrder` (same as KHQR). `required_3ds=false` + no HTML = decided inline.

PCI note: because the cleartext PAN passes through your server before RSA encryption, you are in PCI-DSS scope. To reduce scope, collect card data in a Kardal-hosted page (`createOrder` / hosted link) instead of `directPay`.

## 5. Notify webhook (server-to-server) critical

Kardal POSTs the order result to your `notify_url` (form-encoded), signed with **your** `api_key` . This is the authoritative payment event.

### Payload (key fields)

```
token, out_trade_no, transaction_id, body, total_amount, currency,
status, paid_at, settled_at, settlement_date, expired_at, created_at,
detail[], seller{code,display_name}, bank_ref, trx_id, meta,
refund_histories[], sign_type:"MD5", sign
```

### What your handler MUST do

1. **Verify the sign**: recompute with your `api_key` over the received params (drop `sign` ); reject on mismatch. Never trust an unverified callback.
2. Look up the order by `out_trade_no` ; **check amount + currency** match what you created.
3. **Idempotency**: if already finalized (e.g. status already `SUCCESS` ), do nothing and return 200.
4. Update order status; fulfill (ship goods, etc.) only on `SUCCESS` .
5. Return **HTTP 200**. Any non-2xx → Kardal treats it as failed and **retries** (re-dispatched). Body content is ignored; status code matters.

**Retry & dedupe.** The gateway re-pushes on non-2xx and may push the same terminal status more than once. Your endpoint must be safe to call repeatedly. It skips re-push once it records a successful 200 for a SUCCESS event, but design for at-least-once delivery. Callbacks stop after ~180 days.

Local dev: the gateway must reach your `notify_url` publicly — tunnel it (ngrok) and register that URL. Timeout on Kardal's side is ~25s; respond fast (queue heavy work).

## 6. Query order (reconciliation)

```
POST {base}/api/mch/v2/gateway
{ service:"webpay.acquire.queryOrder", seller_code, out_trade_no, sign_type, sign }
→ data: { status, paid_at, total_amount, currency, payment_detail, card_info, refund_histories[] }
```

Use for: polling during checkout, a fallback if a notify was missed, and end-of-day reconciliation. Source of truth alongside notify.

## 7. Order status lifecycle

| Status                      | Meaning                               | Action                                           |
|-----------------------------|---------------------------------------|--------------------------------------------------|
| WAITING                     | Created, not yet paid                 | Keep polling until paid or expired               |
| SUCCESS                     | Paid & captured <span>terminal</span> | Fulfill the order (once)                         |
| FAILED                      | Payment failed <span>terminal</span>  | Show failure; let buyer retry (new out_trade_no) |
| EXPIRED / CLOSED            | QR/order timed out or closed          | Stop polling; offer retry                        |
| REFUNDED / PARTIAL_REFUNDED | Refund applied                        | Reconcile via refund_histories                   |
| VOID / REVERSAL             | Reversed                              | Reverse fulfillment                              |

## 8. out\_trade\_no & idempotency

- `out_trade_no` = your unique order reference (alphanumeric + dash, ≤32 chars). One per payment attempt.
- It is the join key across create → notify → queryOrder. Generate once, persist before calling the gateway.
- A retried/failed payment needs a **new** `out_trade_no` (duplicates are rejected, `409 DUPLICATED`).

## 9. Errors

| Code                  | Status                              | Common cause                                                                                                          |
|-----------------------|-------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| 400                   | VALIDATION_ERROR                    | Bad sign, missing field, wrong <code>service_code</code> / unprovisioned method ( <i>Invalid payment method bic</i> ) |
| 401                   | UNAUTHENTICATED                     | Expired/invalid token                                                                                                 |
| 403                   | FORBIDDEN                           | IP not whitelisted, account disabled                                                                                  |
| 409                   | DUPLICATED                          | Reused <code>out_trade_no</code>                                                                                      |
| 419                   | EXPIRED                             | Order/QR expired                                                                                                      |
| 422 / 500 / 503 / 504 | PROCESS_FAILED / SYSTEM_ERROR / ... | Downstream/bank issue — retry with backoff, then queryOrder                                                           |

## 10. Reference implementation map

| Concern                                | File (kardal-merchant-demo)                                  |
|----------------------------------------|--------------------------------------------------------------|
| Config / secrets                       | <code>config/kardal.php</code> (+ <code>KARDAL_*</code> env) |
| Token, sign, gateway POST, RSA encrypt | <code>app/Services/Kardal/KardalClient.php</code>            |
| KHQR / Card / query                    | <code>app/Services/Kardal/KardalPaymentService.php</code>    |

|                                            |                                            |
|--------------------------------------------|--------------------------------------------|
| Checkout + status + notify endpoints       | app/Http/Controllers/PaymentController.php |
| Storefront / checkout UI / QR + 3DS render | resources/js/Pages/Store/*                 |

**Note for production:** the demo's `notify()` handler logs + updates status but does not yet verify the inbound sign. Add sign verification (§5 step 1) before going live.